POLITEKNIK NEGERI BENGKALIS
Jurusan Teknik Informatika · Prodi D4 Rekayasa Perangkat Lunak

JOBSHEET PRAKTIKUM
Pengenalan PHPUnit

Mata Kuliah	Pengujian dan Pemeliharaan Perangkat Lunak
Topik	Pengenalan unit testing dengan PHPUnit
Pertemuan ke-	
Alokasi Waktu	3 × 50 menit
Dosen Pengampu	Muhammad Asep Subandri, M.Kom

Nama	Nizrul Hafizam
NIM	6304240041
Kelas	RPL 5 B

A. Tujuan Praktikum

1. Memasang PHPUnit pada proyek PHP menggunakan Composer.
2. Menulis unit test dengan pola Arrange-Act-Assert dan assertion dasar.
3. Menjalankan test serta membaca hasil lulus maupun gagal.
4. Menggunakan data provider untuk menjalankan satu test dengan banyak data.

B. Dasar Teori Singkat

Unit testing adalah pengujian otomatis terhadap bagian terkecil kode, biasanya satu method atau satu class, secara terpisah dari bagian lain. Setiap test disusun dengan pola **Arrange-Act-Assert (AAA)**: siapkan objek dan data (Arrange), jalankan perilaku yang diuji (Act), lalu bandingkan hasilnya dengan harapan (Assert).

PHPUnit adalah framework unit testing standar untuk PHP. Konvensinya: class test berakhiran `Test` dan mewarisi `PHPUnit\Framework\TestCase`; method test bersifat public, diawali kata `test`, dan mengembalikan `void`; hasil diperiksa dengan method assertion seperti `assertSame($harapan, $hasil)`.

Assertion	Memeriksa
`assertSame()`	Nilai dan tipe identik (===). Gunakan sebagai pilihan utama.
`assertEquals()`	Nilai sama (==), tipe boleh berbeda.
`assertTrue()` / `assertFalse()`	Nilai boolean.
`assertStringContainsString()`	Sebuah teks memuat potongan teks tertentu.
`assertInstanceOf()`	Objek bertipe class tertentu.

Data provider adalah method public static yang mengembalikan array data uji. Method test yang ditandai attribute #[DataProvider('namaMethod')] dijalankan sekali untuk setiap baris data.

Contoh kode pada jobsheet ini diadaptasi dari contoh resmi PHPUnit: class Greeter dari manual PHPUnit (bab Writing Tests for PHPUnit) dan class Email dari panduan Getting Started di phpunit.de.

C. Alat dan Bahan

Perangkat	Keterangan
PHP 8.2 atau lebih baru	Cek dengan php -v. Jobsheet ini memakai PHPUnit 11.
Composer 2.x	Cek dengan composer --version.
Text editor	Visual Studio Code atau editor lain.
Terminal dan koneksi internet	Untuk menjalankan perintah dan mengunduh PHPUnit.

Catatan untuk pengguna Windows

Di Command Prompt, tulis vendor\bin\phpunit sebagai ganti vendor/bin/phpunit. Di PowerShell dan Git Bash keduanya bisa dipakai.

D. Langkah Praktikum

Bagian	Kegiatan	Waktu
Praktikum 1	Memasang PHPUnit	25 menit
Praktikum 2	Test pertama: Greeter	25 menit
Praktikum 3	Beberapa assertion: Email	25 menit
Praktikum 4	Data provider	30 menit
Latihan	Latihan mandiri	45 menit

Praktikum 1 – Memasang PHPUnit (25 menit)

1. Buat folder proyek, masuk ke dalamnya, lalu buat composer.json dan pasang PHPUnit:

Terminal

```
mkdir belajar-phpunit
cd belajar-phpunit
composer init --name=polbeng/belajar-phpunit -n
composer require --dev phpunit/phpunit:^11.5
```

2. Buka composer.json dan tambahkan bagian autoload berikut (sejajar dengan require-dev). Dengan classmap, semua class di folder src/ bisa langsung dipakai tanpa require:

composer.json (tambahan)

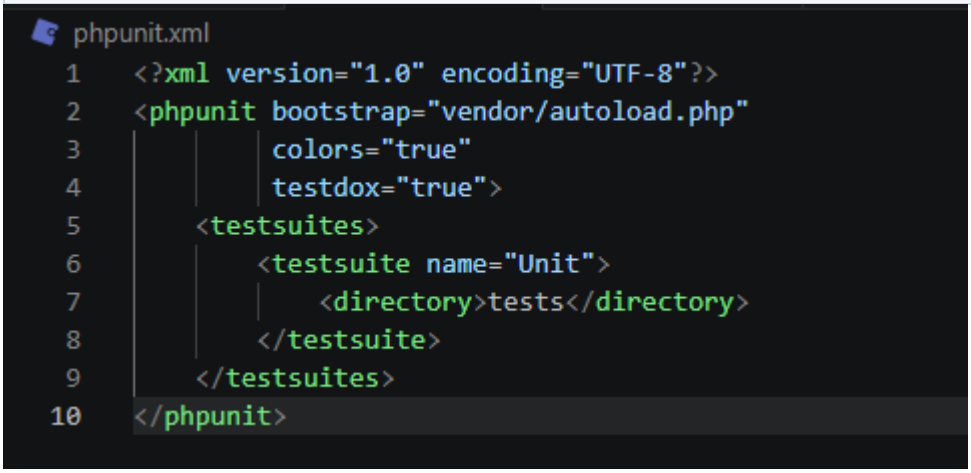
```
"autoload": {
    "classmap": ["src/"]
}
```



3. Buat folder src dan tests, lalu buat file phpunit.xml di root proyek:

phpunit.xml

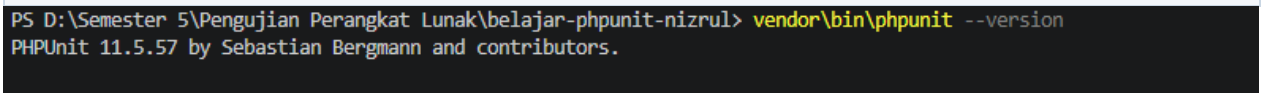
```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit bootstrap="vendor/autoload.php"
         colors="true"
         testdox="true">
    <testsuites>
        <testsuite name="Unit">
            <directory>tests</directory>
        </testsuite>
    </testsuites>
</phpunit>
```



4. Periksa bahwa PHPUnit siap dipakai:

Terminal

```
vendor/bin/phpunit --version
```



Hasil yang diharapkan

Terminal menampilkan PHPUnit 11.5.x. Jika muncul pesan versi PHP tidak memenuhi syarat, perbarui PHP terlebih dahulu.

Praktikum 2 – Test Pertama: Greeter (25 menit)

1. Buat class yang akan diuji:

src/Greeter.php

```php
<?php declare(strict_types=1);

// Class sederhana yang akan diuji: membuat kalimat sapaan
final class Greeter
{
    public function greet(string $name): string
    {
        return 'Hello, ' . $name . '!';
    }
}
```



2. Buat file test. Perhatikan konvensi nama class dan method, serta pola AAA:

tests/GreeterTest.php

```php
<?php declare(strict_types=1);

use PHPUnit\Framework\TestCase;

final class GreeterTest extends TestCase
{
    public function testGreetsWithName(): void
    {
        // Arrange: siapkan objek yang diuji
        $greeter = new Greeter();

        // Act: jalankan perilaku yang diuji
        $greeting = $greeter->greet('Alice');

        // Assert: bandingkan hasil dengan harapan
        $this->assertSame('Hello, Alice!', $greeting);
    }
}
```

tests > GreeterTest.php > ...

```php
<?php declare(strict_types=1);

use PHPUnit\Framework\TestCase;

final class GreeterTest extends TestCase
{
    public function testGreetsWithName(): void
    {
        // Arrange: siapkan objek yang diuji
        $greeter = new Greeter();

        // Act: jalankan perilaku yang diuji
        $greeting = $greeter->greet('Alice');

        // Assert: bandingkan hasil dengan harapan
        $this->assertSame('Hello, Alice!', $greeting);
    }
}
```

3. Perbarui autoloader karena ada class baru, lalu jalankan test:

Terminal

composer dump-autoload
vendor/bin/phpunit

```
PS D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul> composer dump-autoload
Composer could not detect the root package (polbeng/belajar-phpunit) version, defaulting to '1.0.0'. See https://getcomposer.org/root-version
Generating autoload files
Generated autoload files
PS D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul> vendor\bin\phpunit
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.008, Memory: 8.00 MB

Greeter
 ✓ Greets with name

OK (1 test, 1 assertion)
```

Hasil yang diharapkan

Muncul ✓ Greets with name dan baris terakhir OK (1 test, 1 assertion).

4. Di src/Greeter.php, ganti 'Hello, ' menjadi 'Hi, '. Jalankan test lagi dan catat pesan yang muncul. Kembalikan seperti semula.

```
PS D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul> vendor\bin\phpunit
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul\phpunit.xml

F                                                                   1 / 1 (100%)

Time: 00:00.055, Memory: 8.00 MB

Greeter
 ✗ Greets with name

   Failed asserting that two strings are identical.
   --- Expected
   +++ Actual
   @@ @@
   -'Hello, Alice!'
   +'Hi, Alice!'

   D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul\tests\GreeterTest.php:16

FAILURES!
Tests: 1, Assertions: 1, Failures: 1.
```

5. Ganti nama method testGreetsWithName menjadi greetsWithName (tanpa awalan test). Jalankan test, amati hasilnya, lalu kembalikan.

```
PS D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul> vendor\bin\phpunit
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul\phpunit.xml

There was 1 PHPUnit test runner warning:

1) No tests found in class "GreeterTest".

No tests executed!
```

Pertanyaan pengamatan (jawab di laporan)

1. Pada langkah 4, apa arti baris yang diawali tanda - dan + pada pesan gagal?

Jawaban :

Tanda - menunjukkan hasil yang diharapkan (expected), sedangkan tanda + menunjukkan hasil aktual (actual) yang dihasilkan oleh program.

Pada percobaan tersebut, kode Greeter.php diubah dari Hello menjadi Hi, sehingga hasil pengujian tidak sesuai dengan harapan.

2. Pada langkah 5, apa yang terjadi? Mengapa awalan test penting?

Jawaban :

Ketika nama method testGreetsWithName() diubah menjadi greetsWithName(), PHPUnit tidak lagi mengenali method tersebut sebagai pengujian secara otomatis berdasarkan konvensi penamaan.

Akibatnya, method tersebut tidak dijalankan sebagai test dan PHPUnit dapat menampilkan pesan bahwa tidak ada test yang dijalankan.

Awalan test penting karena PHPUnit menggunakan konvensi tersebut untuk mengenali method yang merupakan pengujian. Dengan menggunakan awalan test, method dapat ditemukan dan dijalankan oleh PHPUnit.

3. Tunjukkan bagian Arrange, Act, dan Assert pada test yang Anda tulis.

Jawaban :

a. Arrange (Persiapan)

```
// Arrange: siapkan objek yang diuji
$greeter = new Greeter();
```

Bagian ini digunakan untuk membuat objek dari class Greeter yang akan diuji.

b. Act (Pelaksanaan)

```
// Act: jalankan perilaku yang diuji
$greeting = $greeter->greet('Alice');
```

Bagian ini digunakan untuk menjalankan method greet() dengan parameter nama Alice. Hasilnya disimpan ke variabel $greeting.

c. Assert (Pemeriksaan)

```
// Assert: bandingkan hasil dengan harapan
$this->assertSame('Hello, Alice!', $greeting);
```

Bagian ini digunakan untuk membandingkan hasil aktual dengan hasil yang diharapkan. Jika keduanya sama, pengujian dinyatakan berhasil.

Praktikum 3 – Beberapa Assertion: Email (25 menit)

Class Email hanya bisa dibuat melalui method fromString(), dan akan menolak alamat yang tidak valid. Pada praktikum ini kita hanya menguji alamat yang valid.

1. Buat class Email:

src/Email.php

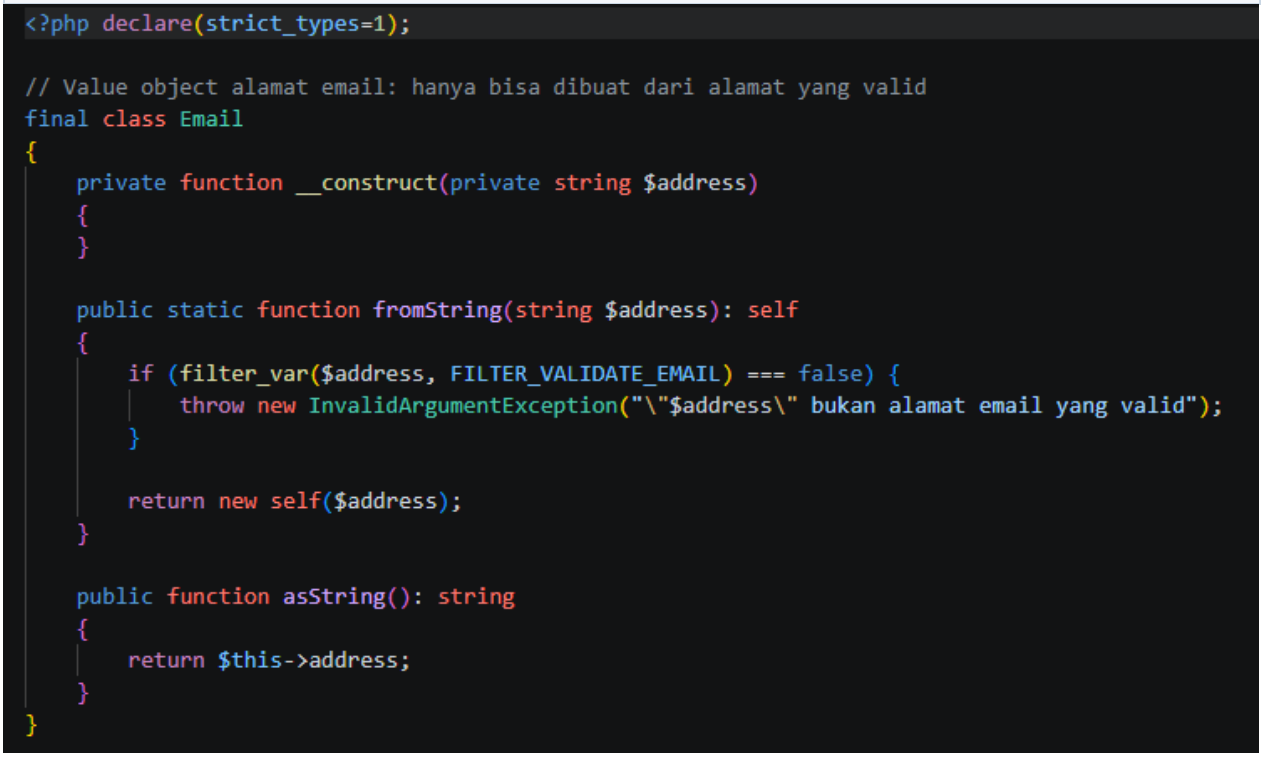
```
<?php declare(strict_types=1);

// Value object alamat email: hanya bisa dibuat dari alamat yang valid
final class Email
{
    private function __construct(private string $address)
    {
    }

    public static function fromString(string $address): self
    {
        if (filter_var($address, FILTER_VALIDATE_EMAIL) === false) {
            throw new InvalidArgumentException("\"$address\" bukan alamat email yang valid");
        }

        return new self($address);
    }

    public function asString(): string
    {
        return $this->address;
    }
}
```



2. Buat test dengan tiga assertion yang memeriksa satu perilaku yang sama:

tests/EmailTest.php

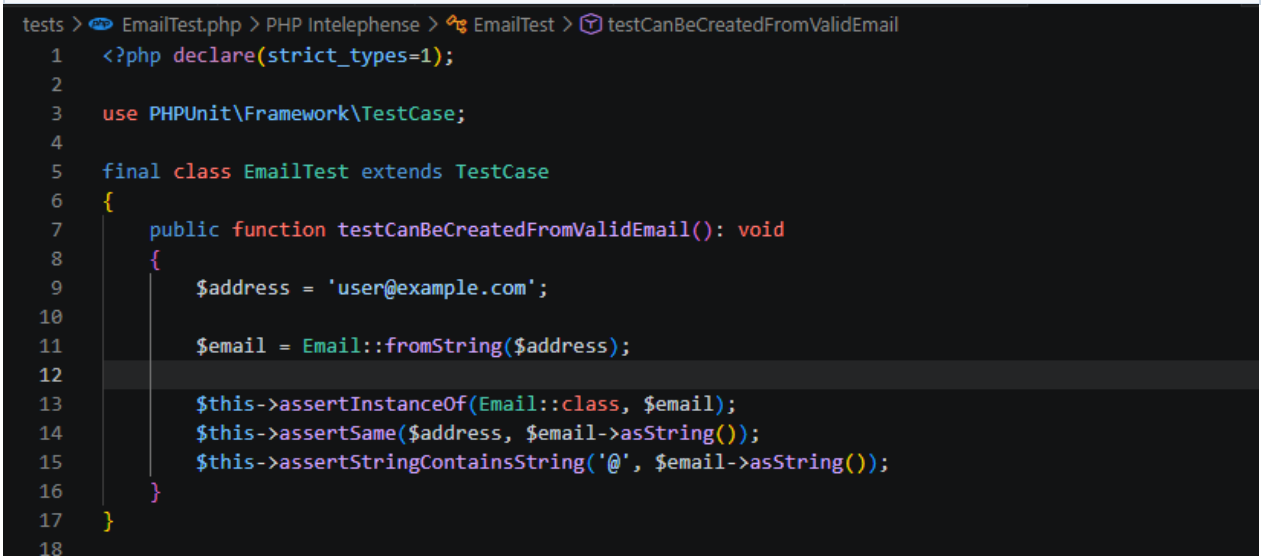
```php
<?php declare(strict_types=1);

use PHPUnit\Framework\TestCase;

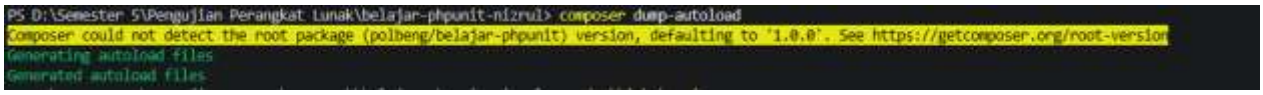
final class EmailTest extends TestCase
{
    public function testCanBeCreatedFromValidEmail(): void
    {
        $address = 'user@example.com';

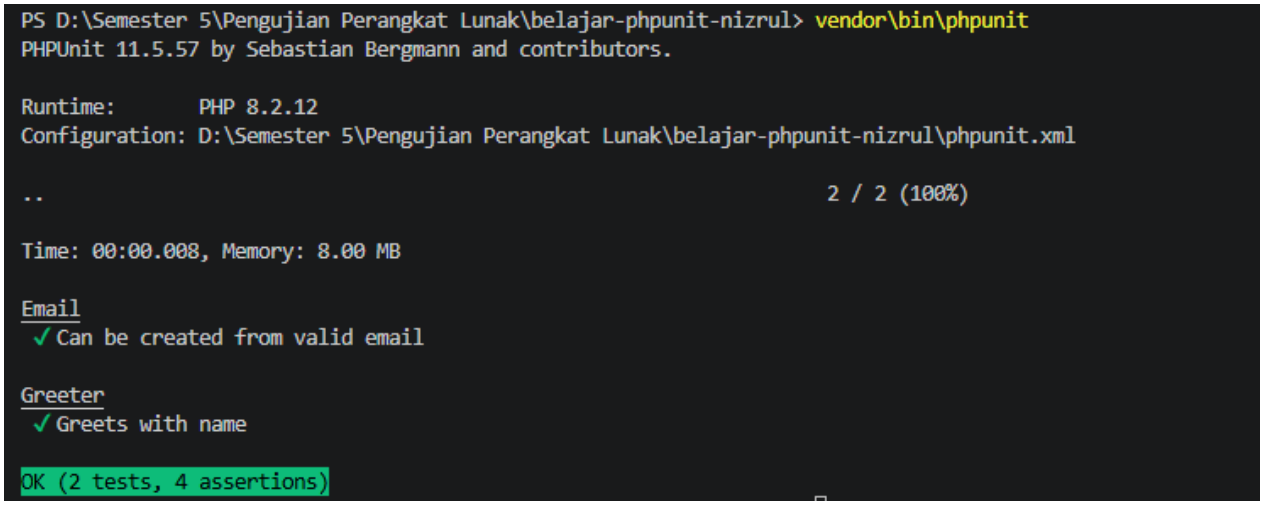
        $email = Email::fromString($address);

        $this->assertInstanceOf(Email::class, $email);
        $this->assertSame($address, $email->asString());
        $this->assertStringContainsString('@', $email->asString());
    }
}
```



3. Jalankan composer dump-autoload, lalu vendor/bin/phpunit.





Hasil yang diharapkan

Terdapat 2 test dari dua class (Greeter dan Email) dan baris terakhir OK (2 tests, 4 assertions).

4. Ubah assertion terakhir menjadi assertStringContainsString('#', ...). Jalankan test, catat hasilnya, lalu kembalikan.

```
tests > EmailTest.php > PHP IntelliSense > EmailTest > testCanBeCreatedFromValidEmail
1   <?php declare(strict_types=1);
2
3   use PHPUnit\Framework\TestCase;
4
5   final class EmailTest extends TestCase
6   {
7       public function testCanBeCreatedFromValidEmail(): void
8       {
9           $address = 'user@example.com';
10
11          $email = Email::fromString($address);
12
13          $this->assertInstanceOf(Email::class, $email);
14          $this->assertSame($address, $email->asString());
15          $this->assertStringContainsString('#', $email->asString());
16      }
17  }
18
```

```
PS D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul> vendor\bin\phpunit
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul\phpunit.xml

F.                                                                  2 / 2 (100%)

Time: 00:00.043, Memory: 8.00 MB

Email
 ✗ Can be created from valid email

   Failed asserting that 'user@example.com' [ASCII](length: 16) contains "#" [ASCII](length: 1).

   D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul\tests\EmailTest.php:15


Greeter
 ✓ Greets with name

FAILURES!
Tests: 2, Assertions: 4, Failures: 1.
```

5. Ubah assertSame($address, ...) menjadi assertEquals($address, ...). Apakah hasilnya berbeda? Kembalikan ke assertSame.

tests > EmailTest.php > PHP IntelliSense > EmailTest > testCanBeCreatedFromValidEmail

```php
<?php declare(strict_types=1);

use PHPUnit\Framework\TestCase;

final class EmailTest extends TestCase
{
    public function testCanBeCreatedFromValidEmail(): void
    {
        $address = 'user@example.com';

        $email = Email::fromString($address);

        $this->assertInstanceOf(Email::class, $email);
        $this->assertEquals($address, $email->asString());
        $this->assertStringContainsString('@', $email->asString());
    }
}

```

```
PS D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul> vendor\bin\phpunit
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul\phpunit.xml

..                                                                  2 / 2 (100%)

Time: 00:00.009, Memory: 8.00 MB

Email
 ✓ Can be created from valid email

Greeter
 ✓ Greets with name

OK (2 tests, 4 assertions)
PS D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul>
```

Pertanyaan pengamatan (jawab di laporan)

1. Mengapa jumlah assertion pada langkah 3 adalah 4, padahal hanya ada 2 test?

Jawaban :

Jumlah test dan jumlah assertion merupakan dua hal yang berbeda. Test adalah metode yang menjalankan pengujian, sedangkan assertion adalah pernyataan yang digunakan untuk memeriksa apakah hasil pengujian sesuai dengan harapan.

Pada langkah 3 terdapat 2 test dengan total 4 assertion karena satu test dapat memiliki lebih dari satu assertion. Contohnya, jika satu test memiliki 3 assertion dan test lainnya memiliki 1 assertion, maka totalnya adalah 4 assertion.

Jadi, jumlah assertion tidak harus sama dengan jumlah test karena satu metode pengujian dapat melakukan beberapa pemeriksaan sekaligus.

2. Jelaskan perbedaan assertSame dan assertEquals beserta contoh kasus ketika hasil keduanya berbeda.

Jawaban :

assertSame() digunakan untuk memeriksa apakah dua nilai benar-benar identik, termasuk tipe datanya. Sedangkan assertEquals() digunakan untuk memeriksa apakah dua nilai memiliki isi atau nilai yang sama tanpa harus memiliki tipe data yang sama. Contohnya, assertSame(10, "10") akan menghasilkan pengujian gagal karena angka 10 bertipe integer, sedangkan "10" bertipe string. Namun, assertEquals(10, "10") akan menghasilkan pengujian lulus karena keduanya dianggap memiliki nilai yang sama. Jadi, perbedaan keduanya terletak pada tingkat ketelitiannya, yaitu assertSame() memeriksa kesamaan nilai sekaligus tipe data, sedangkan assertEquals() lebih berfokus pada kesamaan nilai.

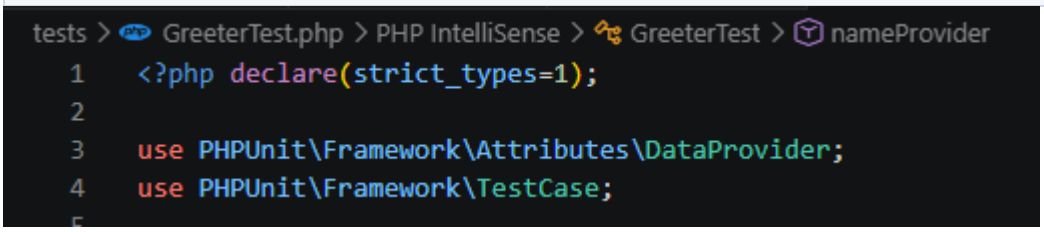
Praktikum 4 – Data Provider (30 menit)

Daripada menulis banyak method yang hampir sama untuk nama yang berbeda, kita gunakan data provider.

1. Tambahkan baris berikut di bawah <?php declare(strict_types=1); pada tests/GreeterTest.php:

tests/GreeterTest.php (tambahan use)

```
use PHPUnit\Framework\Attributes\DataProvider;
```



2. Tambahkan method berikut di dalam class GreeterTest, setelah testGreetsWithName():

tests/GreeterTest.php (tambahan method)

```
#[DataProvider('nameProvider')]
public function testGreetsEachName(string $name, string $expected): void
{
    $greeter = new Greeter();

    $this->assertSame($expected, $greeter->greet($name));
}

// Data provider: setiap baris = satu kasus uji
public static function nameProvider(): array
{
    return [
        'nama pendek'       => ['Ani', 'Hello, Ani!'],
        'nama dengan spasi' => ['Siti Aminah', 'Hello, Siti Aminah!'],
        'huruf kecil'       => ['dedi', 'Hello, dedi!'],
    ];
}
```

tests > GreeterTest.php > PHP IntelliSense > GreeterTest > nameProvider

```php
<?php declare(strict_types=1);

use PHPUnit\Framework\Attributes\DataProvider;
use PHPUnit\Framework\TestCase;

final class GreeterTest extends TestCase
{
    public function testGreetsWithName(): void
    {
        // Arrange: siapkan objek yang diuji
        $greeter = new Greeter();

        // Act: jalankan perilaku yang diuji
        $greeting = $greeter->greet('Alice');

        // Assert: bandingkan hasil dengan harapan
        $this->assertSame('Hello, Alice!', $greeting);
    }

    #[DataProvider('nameProvider')]
    public function testGreetsEachName(string $name, string $expected): void
    {
        $greeter = new Greeter();

        $this->assertSame($expected, $greeter->greet($name));
    }

    // Data provider: setiap baris = satu kasus uji
    public static function nameProvider(): array
    {
        return [
            'nama pendek'       => ['Ani', 'Hello, Ani!'],
            'nama dengan spasi' => ['Siti Aminah', 'Hello, Siti Aminah!'],
            'huruf kecil'       => ['dedi', 'Hello, dedi!'],
            
        ];
    }

}
```

Setiap baris array dipetakan ke parameter method sesuai urutan: elemen pertama ke `$name`, elemen kedua ke `$expected`. Key array (misalnya `'nama pendek'`) menjadi nama kasus di laporan.

3. Jalankan test.

Hasil yang diharapkan

Baris terakhir `OK (5 tests, 7 assertions)`: 1 test Greeter biasa, 3 kasus data provider, dan 1 test Email.

```
PS D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul> vendor\bin\phpunit
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul\phpunit.xml

..                                                                  2 / 2 (100%)

Time: 00:00.010, Memory: 8.00 MB

Email
 ✓ Can be created from valid email

Greeter
 ✓ Greets with name

OK (2 tests, 4 assertions)
PS D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul> vendor\bin\phpunit
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul\phpunit.xml

.....                                                               5 / 5 (100%)

Time: 00:00.020, Memory: 8.00 MB

Email
 ✓ Can be created from valid email

Greeter
 ✓ Greets with name
 ✓ Greets each name with nama pendek
 ✓ Greets each name with nama dengan spasi
 ✓ Greets each name with huruf kecil

OK (5 tests, 7 assertions)
PS D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul>
```

4. Tambahkan baris yang sengaja salah ke `nameProvider()`: `'sengaja salah' => ['Dedi', 'Halo, Dedi!'],`. Jalankan test dan baca laporan kegagalannya.

tests > GreeterTest.php > PHP IntelliSense > GreeterTest > nameProvider

```php
<?php declare(strict_types=1);

use PHPUnit\Framework\Attributes\DataProvider;
use PHPUnit\Framework\TestCase;

final class GreeterTest extends TestCase
{
    public function testGreetsWithName(): void
    {
        // Arrange: siapkan objek yang diuji
        $greeter = new Greeter();

        // Act: jalankan perilaku yang diuji
        $greeting = $greeter->greet('Alice');

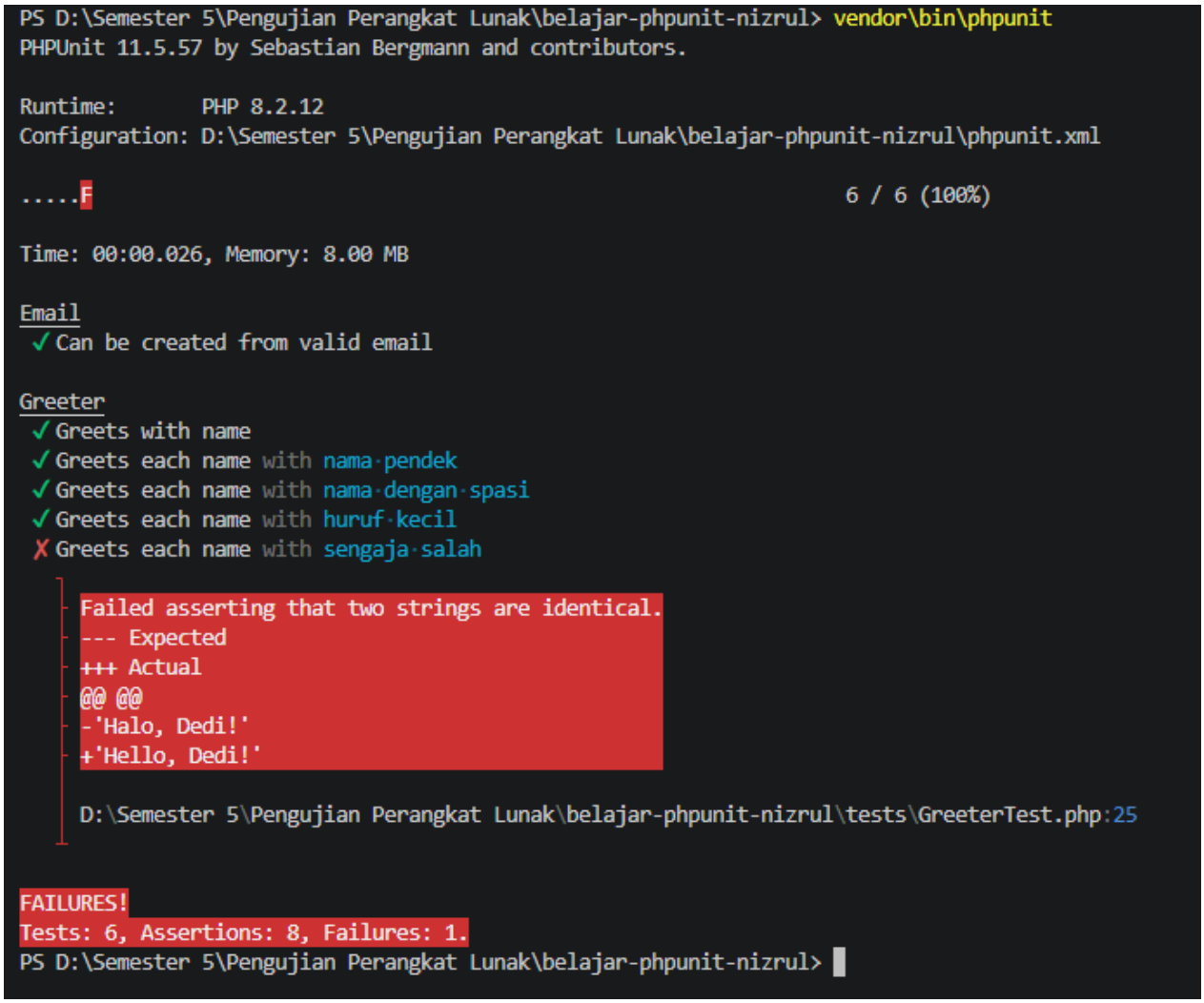
        // Assert: bandingkan hasil dengan harapan
        $this->assertSame('Hello, Alice!', $greeting);
    }

    #[DataProvider('nameProvider')]
    public function testGreetsEachName(string $name, string $expected): void
    {
        $greeter = new Greeter();

        $this->assertSame($expected, $greeter->greet($name));
    }

    // Data provider: setiap baris = satu kasus uji
    public static function nameProvider(): array
    {
        return [
            'nama pendek'       => ['Ani', 'Hello, Ani!'],
            'nama dengan spasi' => ['Siti Aminah', 'Hello, Siti Aminah!'],
            'huruf kecil'       => ['dedi', 'Hello, dedi!'],
            'sengaja salah' => ['Dedi', 'Halo, Dedi!'],
        ];
    }

}
```



5. Hapus kembali baris tersebut dan pastikan semua test lulus.

Pertanyaan pengamatan (jawab di laporan)

1. Bagaimana PHPUnit menunjukkan baris data mana yang gagal pada langkah 4?

Jawaban :

PHPUnit menunjukkan baris data yang gagal melalui pesan error pada hasil pengujian. Pesan tersebut menampilkan nomor baris data (dataset) yang mengalami kegagalan, nilai yang diharapkan, dan nilai aktual yang dihasilkan oleh kode. Dengan begitu, kita dapat mengetahui data mana yang menyebabkan pengujian gagal.

2. Pada langkah 4, apakah yang salah kode Greeter atau data ujinya? Bagaimana Anda memutuskannya?

Jawaban :

Untuk menentukannya, kita perlu membandingkan hasil yang diharapkan dengan hasil yang sebenarnya. Jika kode Greeter tidak menghasilkan output sesuai dengan ketentuan, berarti terdapat kesalahan pada kode. Namun, jika data uji atau hasil yang diharapkan tidak sesuai dengan ketentuan, berarti kesalahan terdapat pada data ujinya. Jadi, keputusan dibuat berdasarkan kesesuaian antara fungsi Greeter dan hasil yang seharusnya.

3. Tambahkan satu baris data baru yang menurut Anda menarik untuk diuji. Jelaskan alasannya.

Jawaban:

Saya akan menambahkan data berupa nama kosong ("") untuk diuji. Alasannya adalah untuk mengetahui apakah fungsi Greeter dapat menangani kondisi ketika pengguna tidak memasukkan nama. Pengujian ini penting agar program tidak menghasilkan sapaan yang tidak sesuai ketika menerima input kosong.

E. Latihan (45 menit)

Latihan 1 – Data provider untuk Email

Di EmailTest, buat method test baru yang memakai data provider untuk memastikan minimal **4 alamat email valid** yang berbeda bentuknya dapat dibuat dan dikembalikan apa adanya oleh asString(). Contoh variasi: alamat dengan titik di nama, angka, subdomain, atau tanda plus.

tests > EmailTest.php > ...

```php
<?php declare(strict_types=1);

use PHPUnit\Framework\TestCase;
use PHPUnit\Framework\Attributes\DataProvider;

final class EmailTest extends TestCase
{
    public function testCanBeCreatedFromValidEmail(): void
    {
        $address = 'user@example.com';

        $email = Email::fromString($address);

        $this->assertInstanceOf(Email::class, $email);
        $this->assertSame($address, $email->asString());
        $this->assertStringContainsString('@', $email->asString());
    }

    #[DataProvider('validEmailProvider')]
    public function testCanCreateDifferentValidEmails(string $address): void
    {
        $email = Email::fromString($address);

        $this->assertSame($address, $email->asString());
    }

    public static function validEmailProvider(): array
    {
        return [
            'email biasa' => ['user@example.com'],
            'email dengan titik' => ['nama.depan@example.com'],
            'email dengan angka' => ['user123@example.com'],
            'email subdomain' => ['user@mail.example.com'],
            'email dengan plus' => ['user+test@example.com'],
        ];
    }

}

```

```
PS D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul> vendor\bin\phpunit
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: D:\Semester 5\Pengujian Perangkat Lunak\belajar-phpunit-nizrul\phpunit.xml

..........                                                        10 / 10 (100%)

Time: 00:00.016, Memory: 8.00 MB

Email
 ✓ Can be created from valid email
 ✓ Can create different valid emails with email biasa
 ✓ Can create different valid emails with email dengan titik
 ✓ Can create different valid emails with email dengan angka
 ✓ Can create different valid emails with email subdomain
 ✓ Can create different valid emails with email dengan plus

Greeter
 ✓ Greets with name
 ✓ Greets each name with nama pendek
 ✓ Greets each name with nama dengan spasi
 ✓ Greets each name with huruf kecil

OK (10 tests, 12 assertions)
```

Latihan 2 – Siapa yang menangkap bug?

Ganti isi method greet() satu per satu dengan versi berikut. Untuk setiap versi, jalankan test dan catat test atau kasus data provider mana yang gagal. Kembalikan kode asli setelah selesai.

Versi	Isi baris return
A	return 'Hello ' . $name . '!'; (koma hilang)

src > Greeter.php > ...

```php
<?php declare(strict_types=1);

// Class sederhana yang akan diuji: membuat kalimat sapaan
final class Greeter
{
    public function greet(string $name): string
    {
        return 'Hello ' . $name . '!';
    }
}
```

Versi	Isi baris return
B	return 'Hello, ' . ucfirst($name) . '!'; (huruf awal dijadikan kapital)
C	return 'Hello, ' . trim($name) . '!'; (spasi di tepi dibuang)

src > Greeter.php > PHP IntelliSense > Greeter > greet

```php
<?php declare(strict_types=1);

// Class sederhana yang akan diuji: membuat kalimat sapaan
final class Greeter
{
    public function greet(string $name): string
    {
        return 'Hello, ' . ucfirst($name) . '!';
    }
}
```

src > Greeter.php > PHP Intelephense > Greeter > greet

```php
<?php declare(strict_types=1);

// Class sederhana yang akan diuji: membuat kalimat sapaan
final class Greeter
{
    public function greet(string $name): string
    {
        return 'Hello, ' . trim($name) . '!';
    }
}
```

Pertanyaan pengamatan (jawab di laporan)

1. Versi mana yang tidak tertangkap oleh test yang ada? Mengapa?

Jawaban :

Versi yang tidak tertangkap oleh pengujian yang ada adalah ketika nama berisi spasi kosong (" "). Hal ini karena test yang tersedia mungkin hanya menguji nama normal atau string kosong (""), sehingga belum memeriksa kondisi ketika pengguna memasukkan spasi tanpa karakter. Akibatnya, program mungkin tetap menganggap input tersebut sebagai nama yang valid.

2. Tambahkan satu baris data ke nameProvider() agar versi tersebut tertangkap.

Jawaban :

```php
return [
    'nama pendek'       => ['Ani', 'Hello, Ani!'],
    'nama dengan spasi' => ['Siti Aminah', 'Hello, Siti Aminah!'],
    'huruf kecil'       => ['dedi', 'Hello, dedi!'],
    'nama dengan spasi di depan' => [' Ani', 'Hello,  Ani!'],
    'nama dengan spasi di akhir' => ['Ani ', 'Hello, Ani !'],
];
```

F. Tugas Mandiri

Buat class KonversiSuhu di folder src dengan dua method berikut, lalu tulis test-nya di tests/KonversiSuhuTest.php:

Method	Rumus
celsiusKeFahrenheit(float $c): float	F = C × 9/5 + 32
celsiusKeKelvin(float $c): float	K = C + 273,15

- Gunakan data provider dengan minimal 4 kasus untuk setiap method, misalnya titik beku, titik didih, suhu negatif, dan suhu tubuh.
- Semua test memakai pola AAA dan nama method yang menjelaskan perilaku.
- Jalankan vendor/bin/phpunit dan pastikan semua test lulus.

Petunjuk

Coba uji 36,6 °C dengan assertSame(97.88, ...). Jika gagal padahal rumus benar, cari tahu penyebabnya, lalu gunakan assertEqualsWithDelta(97.88, $hasil, 0.001). Jelaskan temuan ini di laporan.

Code Program

Folder SRC

KonversiSuhu.php

src > KonversiSuhu.php > ...

```php
<?php declare(strict_types=1);

final class KonversiSuhu
{
    public function celsiusKeFahrenheit(float $c): float
    {
        return $c * 9 / 5 + 32;
    }

    public function celsiusKeKelvin(float $c): float
    {
        return $c + 273.15;
    }
}

```

Floder Test

KonversiSuhuTest.php

```php
<?php declare(strict_types=1);

use PHPUnit\Framework\TestCase;
use PHPUnit\Framework\Attributes\DataProvider;

final class KonversiSuhuTest extends TestCase
{
```

```
    #[DataProvider('fahrenheitProvider')]
    public function testCelsiusCanBeConvertedToFahrenheit(
        float $celsius,
        float $expected
    ): void {
        // Arrange
        $konversi = new KonversiSuhu();

        // Act
        $hasil = $konversi->celsiusKeFahrenheit($celsius);

        // Assert
        $this->assertEqualsWithDelta($expected, $hasil, 0.001);
    }

    public static function fahrenheitProvider(): array
    {
        return [
            'titik beku air' => [0, 32],
            'titik didih air' => [100, 212],
            'suhu negatif' => [-40, -40],
            'suhu tubuh' => [36.6, 97.88],
        ];
    }

    #[DataProvider('kelvinProvider')]
    public function testCelsiusCanBeConvertedToKelvin(
        float $celsius,
        float $expected
    ): void {
        // Arrange
        $konversi = new KonversiSuhu();

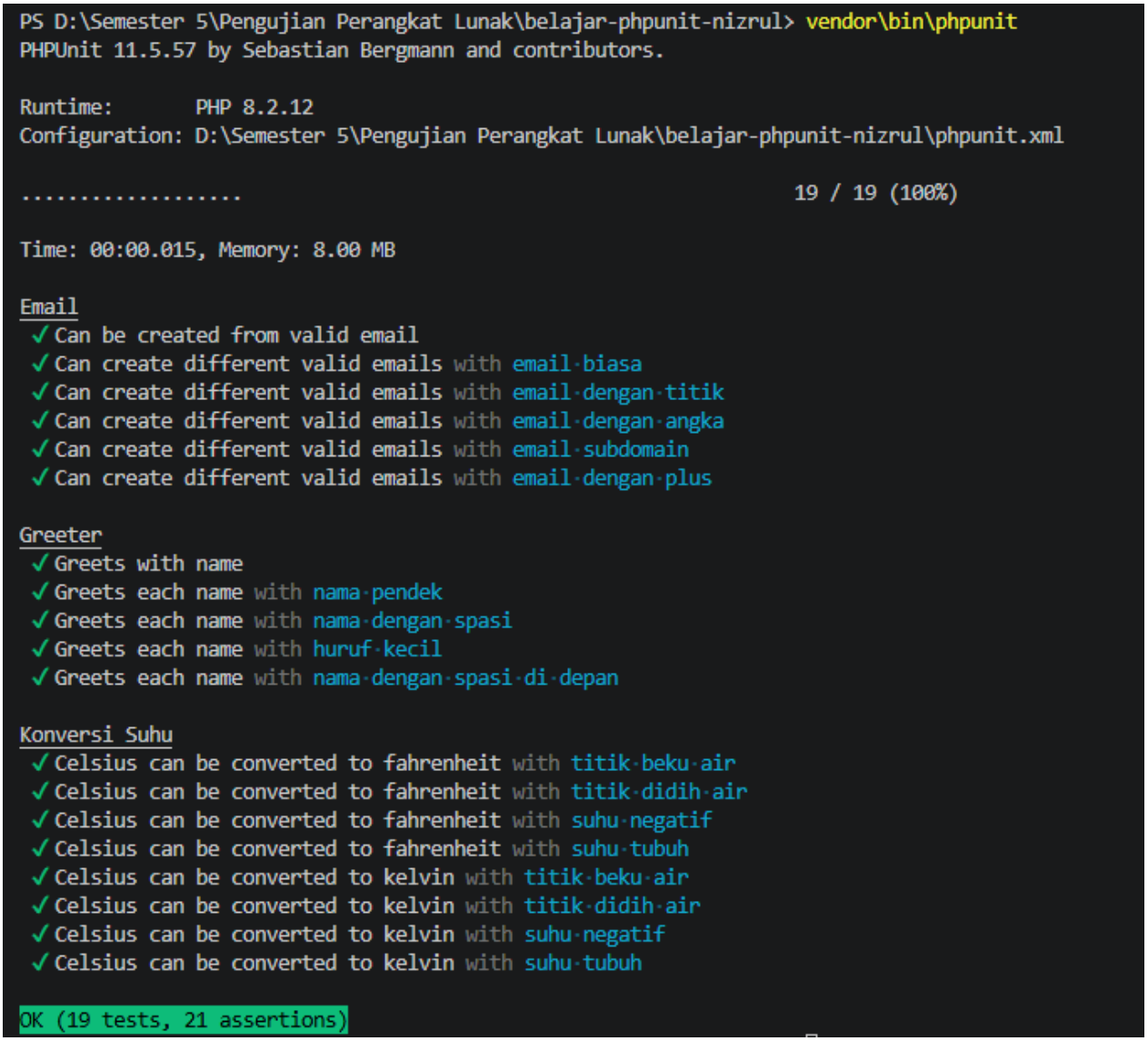
        // Act
        $hasil = $konversi->celsiusKeKelvin($celsius);

        // Assert
        $this->assertEqualsWithDelta($expected, $hasil, 0.001);
    }

    public static function kelvinProvider(): array
    {
        return [
            'titik beku air' => [0, 273.15],
            'titik didih air' => [100, 373.15],
            'suhu negatif' => [-40, 233.15],
            'suhu tubuh' => [36.6, 309.75],
        ];
    }
}
```

Output





Kesimpulan :

Dari praktikum pengenalan PHPUnit, saya mendapatkan pemahaman tentang cara melakukan pengujian otomatis pada kode PHP untuk memastikan program menghasilkan output yang sesuai. Saya juga mempelajari fungsi assertion seperti assertSame() dan assertEquals(), serta Data Provider yang memungkinkan pengujian menggunakan beberapa variasi data. Selain itu, saya menjadi lebih memahami cara membaca hasil pengujian dan menemukan kesalahan (bug) pada program. Praktikum ini memberikan pemahaman bahwa unit testing berperan penting dalam meningkatkan kualitas kode dan meminimalkan kesalahan dalam program.

G. Laporan

Kumpulkan laporan dalam format PDF yang berisi:

1. Tautan repositori Git berisi kode praktikum, latihan, dan tugas.
2. Tangkapan layar hasil vendor/bin/phpunit di akhir setiap praktikum dan tugas.
3. Jawaban seluruh pertanyaan pengamatan.
4. Kesimpulan singkat tentang apa yang Anda pelajari.

H. Referensi

Bergmann, S. PHPUnit Manual: Writing Tests for PHPUnit. https://docs.phpunit.de/

PHPUnit. Getting Started with PHPUnit. https://phpunit.de/getting-started/